Solo Enterprise for agentgateway: deployment models on shared Kubernetes

Where to place agentgateway when a platform team owns shared clusters and application teams are confined to their own namespaces. Three in-cluster models, then the multi-cluster question without a Solo mesh. The right answer depends on who should own routing and policy, how many clusters share a region, and how much the platform team wants to review; use this as the starting point for that discussion.

FIXED IN EVERY MODEL

- **One controller per cluster, cluster-scoped by design.** Controller, CRDs, the `GatewayClass` (fixed name `enterprise-agentgateway`) and the controller's ClusterRoles are installed once per cluster by the platform owner; one reviewable `helm template`. There is no multi-cluster control plane; clusters stay identical through GitOps (Argo CD is documented).
- **One proxy per `Gateway` object**, deployed in that object's namespace. Routes (`HTTPRoute`), backends and policies attach from the same or other namespaces under Gateway API rules (`allowedRoutes`, `ReferenceGrant`).
- **Permissions today:** the controller reads Secrets and ConfigMaps cluster-wide and manages extension RBAC at runtime; `rbac.gatewayNamespaces` and `discoveryNamespaceSelectors` limit what it watches and where it deploys, not what it may read. Namespace-scoped reads and a formal same-cluster multi-tenancy model are on the engineering roadmap. Unused extensions (ext-auth, WAF, rate limit, ext-cache) can be disabled today, which removes their roles.
- **Hosted model providers are external hostnames from every cluster**; only in-cluster MCP servers and self-hosted models are local to a gateway.

THREE IN-CLUSTER MODELS

MODEL	PROS	CONS
A. Gateway in a dedicated platform namespace; the gateway team owns gateway, routes and policies; routes to application namespaces	Smallest review surface (one namespace, one team, one proxy fleet). Policy, quotas, provider secrets and audit in one place. Application teams stay decoupled from the gateway APIs; the platform stays opinionated. Simplest fit with namespace-scoped controller permissions.	Every route or policy change goes through the gateway team (a ticket queue as adoption grows). A `ReferenceGrant` in each application namespace whose Services the gateway routes to (Gateway API rule for cross-namespace backends). One fleet: a bad policy affects all teams.
B. Gateway in each application namespace	Strongest isolation; fits a one-namespace-per-team rule exactly; no cross-namespace references; teams move at their own pace.	N proxy fleets to size, upgrade and monitor; N copies of provider secrets; central policy (PII masking, budgets, audit shape) re-applied per namespace and prone to drift; the controller needs reach into every namespace with a `Gateway`; the weakest story for a security reviewer.
C. Centralized gateway, configuration owned by application namespaces (route delegation)	One fleet and one platform-policy point with no ticket queue: the platform owns the `Gateway` and parent routes, teams own child routes, backends and policies in their namespaces, platform policies are inherited. Matches the Gateway API ownership model; the shape Solo's agentregistry integration uses; scales with teams.	Two-level ownership to document (inherited vs team-settable). The controller watches the application namespaces, so it reads their routes, backends and referenced Secrets. Teams are coupled to the gateway APIs and need to learn them.

ACROSS CLUSTERS, WITHOUT A SOLO MESH

Existing Istio east-west gateways are fine for connectivity between clusters. What is missing without Solo's Istio multi-cluster is service discovery that makes a remote Service look local, so a gateway that serves several clusters routes to a declared inventory. Two shapes:

	REGIONAL ROUTING TIER (ONE GATEWAY PER REGION, A FEW WORKLOAD CLUSTERS BEHIND IT)	GATEWAY PER CLUSTER
Backends	Hosted providers as external hostnames; workload-cluster MCP servers and models as declared backends (static hostname backends) over the east-west gateways. No discovery: the inventory lives in GitOps.	Local Services and external hostnames.
Pros	One entry point and one audit location per region; fewer controllers to review and upgrade; the workload clusters' core platform is untouched; a documented Solo gateway deployment pattern (central ingress tier plus a second tier).	Smallest permission footprint per cluster; failures contained to one cluster; no hairpin.
Cons	The hairpin through the east-west path is fine at low latency inside a region and gets harder as clusters, zones and regions multiply; static backend list; one controller's read scope spans several clusters' teams; region-wide blast radius.	N installs and N RBAC reviews; a gateway inside every workload cluster touches the core platform; per-team quota is per cluster unless a shared rate-limit or budget store is used.
Later	Solo's Istio ambient multi-cluster turns the declared list into `*.mesh.internal` global services and allows a workload-cluster waypoint for service-to-service L7 policy without the hairpin. Failover is the same in both shapes: LLM priority groups and health eviction prefer the same-region provider deployment and spill to the next; global DNS picks the regional entry point at the client edge.	

RECOMMENDATION

1. **In-cluster: start with A, grow into C.** A gives the smallest review surface and keeps application teams off the gateway APIs while the platform team learns the product. When the ticket queue becomes the bottleneck, add a parent route and delegate (C): it is additive, nothing is thrown away, and it is the shape Solo's own agentregistry gateway integration uses. B suits only organizations without a central policy requirement.
2. **Across clusters: a regional routing tier when a region is a few clusters on a low-latency network and one entry point matters; a gateway per cluster when permission footprint and failure containment matter more, or clusters are far apart.** Mixing is normal: a routing tier for ingress and egress, plus a gateway inside a workload cluster only where service-to-service policy must not hairpin.
3. **Discovery** is a mesh feature, not a gateway feature. Add Solo's Istio multi-cluster when the declared inventory stops scaling.
4. **Do now, in any model:** enumerate `rbac.gatewayNamespaces`, set `discoveryNamespaceSelectors` to the team namespaces, disable unused extensions, and audit and alert on the controller's Secret reads.

Questions that change the answer: clusters per region and the latency between them; any air-gapped boundary (a regional tier cannot cross it); whether application teams should author their own routes and policies, and when. Docs: docs.solo.io/agentgateway/kubernetes/latest/documentation/ (traffic-management/route-delegation, security/rbac-hardening, llm/rbac, llm/failover).